ANEXO 03

# ESQUEMA DE LA BASE DE DATOS

Plataforma de Gestión de Información de Privacidad (PIMS) — GESTITEL E.I.R.L.

Documentación técnica de base de datos — Documento 03

*Versión del sistema documentada: migraciones core/0001 – core/0043 · customer/0001 – customer/0004*

Arequipa, Perú — Octubre 2026

# TABLA DE CONTENIDO

A quién va dirigido ..... 3
Qué contiene ..... 3
**1. Motor, organización física y modelo multiempresa** ..... 3
1.1 Por qué PostgreSQL es obligatorio ..... 3
1.2 Organización de esquemas ..... 4
1.3 Consecuencias prácticas del modelo ..... 4
**2. Llaves primarias** ..... 4
**3. Llaves foráneas y reglas de borrado** ..... 5
3.0 Dónde se aplican realmente estas reglas — advertencia para administradores de base de datos ..... 5
3.1 PROTECT — la regla por defecto de la auditoría ..... 6
3.2 CASCADE — inventario completo ..... 7
3.3 SET_NULL — inventario completo ..... 8
**4. Restricciones de unicidad** ..... 9
4.1 Unicidad de una sola columna ..... 9
4.2 Unicidad compuesta ..... 9
4.3 Restricciones únicas parciales — «solo uno activo» ..... 10
**5. Índices** ..... 10
**6. Borrado lógico y su interacción con las restricciones UNIQUE** ..... 11
6.1 Cómo funciona el borrado lógico ..... 11
6.2 La consecuencia que hay que conocer ..... 12
6.3 Tablas sin borrado lógico ..... 12
**7. Tablas con comportamiento especial** ..... 13
7.1 Tablas singleton — core_systemsettings y customer_platformaisettings ..... 13
7.2 Tablas de historial inmutable ..... 13
7.3 Catálogos provisionados automáticamente ..... 14
**8. Tipos de datos empleados** ..... 14
8.1 Dos criterios que conviene destacar ..... 15
**9. Tablas nativas de Django** ..... 15
**10. Evolución del esquema** ..... 16
Documentos relacionados ..... 16

### A quién va dirigido

A personal técnico que necesite conocer **la estructura y las reglas de integridad** de la base de datos: administradores de base de datos, equipos de integración o migración, auditores técnicos.

Presupone familiaridad con conceptos de bases de datos relacionales (llave primaria, llave foránea, restricción, índice). Para el detalle campo por campo, consulte el documento **02 — Diccionario de datos**; para el mapa visual, el **01 — Diagramas entidad-relación**.

### Qué contiene

1. Motor, organización física y modelo multiempresa.
2. Llaves primarias: convención única en todo el sistema.
3. Llaves foráneas y las tres reglas de borrado, con el inventario completo.
4. Restricciones de unicidad, incluidas las **parciales** y las **compuestas**.
5. Índices.
6. **El borrado lógico y su interacción con las restricciones** `UNIQUE` — la sutileza que más incidentes ha causado en este sistema.
7. Tablas con comportamiento especial: singleton e inmutables.
8. Tipos de datos empleados y criterios de elección.
9. Tablas nativas de Django.
10. Evolución del esquema: cómo se versiona.

## 1. Motor, organización física y modelo multiempresa

| Aspecto | Valor |
|---|---|
| Motor | **PostgreSQL** (obligatorio, no es intercambiable) |
| Capa de acceso | Django 6 ORM con la extensión `django-tenants` |
| Codificación | UTF-8 |
| Zona horaria de almacenamiento | **UTC**. Las conversiones a hora de Perú (`America/Lima`) se hacen solo al generar reportes |
| Bases de datos | **Una sola** |
| Esquemas | `public` + **uno por empresa cliente** |

### 1.1 Por qué PostgreSQL es obligatorio

El aislamiento entre empresas se implementa con **esquemas de PostgreSQL**, una característica que no tiene equivalente directo en otros motores. Además se usan **restricciones únicas parciales** (`UNIQUE ... WHERE`), soportadas por PostgreSQL pero no por todos los motores, y el tipo **`jsonb`**.

### 1.2 Organización de esquemas

```
Base de datos PostgreSQL
│
├── esquema  public
│   ├── customer_client        ← catálogo de empresas
│   ├── customer_domain        ← dominios de acceso
│   ├── customer_license       ← licencias por días
│   ├── customer_platformlog   ← bitácora de superadministradores
│   ├── customer_platformaisettings ← configuración global del asistente (fila única)
│   ├── auth_user, auth_group, auth_permission   ← superadministradores de Gestitel
│   └── django_migrations, django_session, django_content_type, django_admin_log
│
├── esquema  <empresa_1>
│   ├── 47 tablas de la aplicación (core_*)
│   ├── 3 tablas puente de relaciones muchos a muchos
│   └── auth_user, auth_group, auth_permission, ...   ← usuarios y roles DE ESA EMPRESA
│
├── esquema  <empresa_2>
│   └── (estructura idéntica, datos independientes)
│
└── ... un esquema por cada empresa
```

### 1.3 Consecuencias prácticas del modelo

**El mismo nombre de tabla existe N veces**, una por esquema. `core_asset` no identifica una tabla, identifica una tabla *dentro de un esquema*. En consultas manuales hay que calificarla: `SELECT * FROM empresa_a.core_asset;`.

**No existe ninguna llave foránea entre esquemas.** `customer_client` no referencia tablas de empresa ni al revés. El único vínculo es el texto de `customer_client.schema_name`. Eliminar esa posibilidad física es lo que hace que una consulta mal escrita no pueda mezclar datos de dos empresas.

**Cada empresa tiene sus propios usuarios y permisos.** Las tablas `auth_user`, `auth_group` y `auth_permission` se replican por esquema. Un usuario de la empresa A no existe en la empresa B. La única excepción son los superadministradores de Gestitel, que viven en `public` y se proyectan en cada esquema mediante un mecanismo de usuario espejo (ver documento **05**).

**Las migraciones se aplican a todos los esquemas.** Añadir una columna la añade en las N copias, dentro de una transacción por esquema.

**Al dar de alta una empresa, su esquema se crea y se migra automáticamente**. Como parte de ese proceso se siembran los datos iniciales: los 93 controles del Anexo A, las tres matrices de evaluación, las cuatro franjas de tasación, los catálogos configurables, el periodo inicial de riesgos, las 93 filas de la Declaración de Aplicabilidad y la fila única de ajustes.

## 2. Llaves primarias

**Convención única, sin excepciones:** las 52 tablas tienen una llave primaria llamada `id`, de tipo **`bigserial`** (entero de 64 bits autoincremental).

```
id bigserial NOT NULL PRIMARY KEY
```

No se usan llaves primarias compuestas ni naturales (códigos de negocio) en ninguna tabla.

**Los códigos de negocio son columnas aparte con restricción de unicidad**, no llaves primarias: `core_asset.code`, `core_document.code`, `core_process.code`, `core_consent.consent_code`, `core_incident.incident_code`, `core_audit.audit_code`, `core_arcorequest.case_code`, `core_training.training_code`, `core_certificate.code`, `core_annexacontrol.code`, `core_systemcatalog.code`, `core_riskperiod.name`, `core_gaprequirement.requirement_code`.

**Por qué.** Un código de negocio puede cambiar de formato o de reglas de generación; una llave primaria propagada a decenas de tablas hijas, no. La separación deja el código libre de evolucionar sin reescribir relaciones. Dos casos lo ilustran: `core_riskevaluation.risk_code` es único **por periodo** y no globalmente, y `core_asset.code` sigue la nomenclatura automática `ACT-<abreviatura de categoría>-<correlativo>`. Ninguno de los dos participa en las relaciones: riesgos, controles y documentos apuntan al `id` interno, de modo que el código puede cambiar de regla sin desconectar nada. El código previo de cada activo se conserva en `core_asset.legacy_code`, consultable desde el buscador del Inventario.

**Excepción documentada:** `core_systemsettings` (una por empresa) y `customer_platformaisettings` (una sola, en `public`) también tienen `id bigserial`, pero su valor **se fuerza siempre a 1** desde la capa de aplicación, de modo que la tabla nunca puede contener más de una fila (apartado 7.1).

## 3. Llaves foráneas y reglas de borrado

El sistema declara **108 llaves foráneas** (102 en las tablas de empresa y 6 en las de plataforma; recuento por introspección del modelo en ejecución) con tres reglas de borrado, elegidas caso a caso:

| Regla | Cantidad | Qué hace | Cuándo se usa |
|---|---|---|---|
| `PROTECT` | 68 | Impide borrar el padre mientras exista un hijo que lo referencia. Error de integridad. | Datos que son evidencia, y la trazabilidad de auditoría. |
| `CASCADE` | 27 | Al borrar el padre se borran automáticamente los hijos. | Hijos que no tienen sentido separados del padre. |
| `SET_NULL` | 13 | Al borrar el padre, la referencia del hijo queda en `NULL`; el hijo sobrevive. | Referencias que son contexto, no sustancia. |

### 3.0 Dónde se aplican realmente estas reglas — advertencia para administradores de base de datos

> **Las tres reglas anteriores las aplica la capa de aplicación, no el motor PostgreSQL.**

Esto es un comportamiento del marco de trabajo (Django) y conviene conocerlo antes de operar sobre la base de datos directamente. Verificado sobre la instalación real: **las 245 restricciones de llave foránea existentes están declaradas sin cláusula `ON DELETE`**, es decir, con el comportamiento por omisión `NO ACTION`, y las 245 son `DEFERRABLE INITIALLY DEFERRED`:

```
-- Definición real de cualquier llave foránea del sistema:
ALTER TABLE core_riskevaluationasset
  ADD CONSTRAINT core_riskevaluationasset_asset_id_..._fk_core_asset_id
  FOREIGN KEY (asset_id) REFERENCES core_asset(id)
  DEFERRABLE INITIALLY DEFERRED;
-- Sin ON DELETE CASCADE, sin ON DELETE SET NULL, sin ON DELETE RESTRICT.
```

> **De dónde sale la cifra y por qué no es fija.** «245» es el recuento directo del catálogo de PostgreSQL (`pg_constraint`, consulta del documento **04**, apartado 7) sobre **toda** la base de la instalación de referencia, que tiene **dos empresas**: **115 por cada esquema de empresa** (108 en las tablas `core_*` —las 102 que declara el modelo más 6 de las tres tablas puente— y 7 en las tablas nativas de Django) **+ 15 en `public`** (6 en las tablas `customer_*` y 9 en las nativas). **Cada empresa nueva añade 115.** Lo que no cambia es la propiedad verificada: todas `NO ACTION`, todas diferibles.

Cuando se elimina un registro **a través de la aplicación**, es el ORM el que recorre las relaciones y ejecuta, dentro de una transacción, los `DELETE` en cascada, los `UPDATE ... SET NULL` o el rechazo de la operación, según corresponda.

**Consecuencias prácticas:**

| Vía de acceso | Qué ocurre |
|---|---|
| Aplicación (interfaz web, API) | Las tres reglas se aplican con exactitud. |
| `DELETE` manual en SQL sobre una fila padre | **`NO ACTION` lo impide siempre**, sea cual sea la regla declarada: PostgreSQL rechaza el borrado mientras exista cualquier hijo que la referencie. No hay cascada automática ni puesta a nulo. |

Dicho de otro modo: **desde SQL directo el motor es más restrictivo que la aplicación, no menos.** No existe el riesgo de que un borrado manual arrase con filas hijas por una cascada silenciosa; el riesgo es el contrario, que la sentencia falle. La forma correcta de eliminar datos es siempre la aplicación.

`DEFERRABLE INITIALLY DEFERRED` significa que la comprobación de integridad se posterga al final de la transacción, lo que permite reordenar filas relacionadas dentro de una misma operación. **No es solo una curiosidad teórica:** la migración `core/0042` lo **aprovecha en la práctica** para reordenar físicamente el `id` de `core_annexacontrol` y de `core_gapannexaassessment` con un «puente de dos saltos» (mover cada fila a un `id` temporal muy alto y de ahí a su destino final, dentro de la misma transacción) sin violar ninguna llave foránea a mitad de camino.

## 3.1 PROTECT — la regla por defecto de la auditoría

De las 68 llaves `PROTECT`, **58 son columnas de auditoría** (`created_by_id`, `updated_by_id`, `deleted_by_id`) que apuntan a `auth_user`. El efecto buscado es directo: **un usuario que haya creado o modificado cualquier registro no puede eliminarse de la base de datos**. La trazabilidad de «quién hizo qué» no se puede romper borrando al autor.

Las diez restantes protegen datos de negocio:

| Llave foránea | Protege |
|---|---|
| `core_asset.process_id → core_process` | Un proceso con activos asociados. |
| `core_riskevaluationasset.asset_id → core_asset` | Un activo referenciado por un riesgo. |
| `core_riskevaluation.period_id → core_riskperiod` | Un periodo que contiene riesgos. |
| `core_gapannexaassessment.annex_a_control_id → core_annexacontrol` | Un control del catálogo del Anexo A evaluado en la Declaración de Aplicabilidad. |
| `core_risktreatment.annex_a_control_id → core_annexacontrol` | Un control del Anexo A propuesto como tratamiento. |
| `core_document.approved_by_id → auth_user` | El usuario que aprobó formalmente un documento. |

| Llave foránea | Protege |
|---|---|
| `customer_platformlog.user_id → auth_user` | El superadministrador que ejecutó una acción de plataforma. |
| `core_personaldataimpactassessment.asset_id → core_asset` | Un activo que tiene su EIPD (relación uno a uno). |
| `core_personaldataoperation.assessment_id → core_personaldataimpactassessment` | Una EIPD con operaciones ya registradas. |
| `core_personaldataoperation.annuls_id → core_personaldataoperation` (autorreferencia) | Una operación que ya anuló a otra: no se puede romper la cadena de corrección borrando el eslabón intermedio. |

**Advertencia importante sobre `PROTECT` y borrado lógico.** En el flujo normal de la aplicación estas protecciones **no llegan a dispararse**, porque la aplicación no ejecuta `DELETE`: marca `is_deleted = true` (apartado 6). Una fila marcada como eliminada sigue existiendo y sigue referenciando a su padre, de modo que `PROTECT` nunca entra en juego.

Por eso las reglas de negocio equivalentes **se comprueban explícitamente en la capa de aplicación**, que cuenta los hijos *vivos* y responde con un error explicativo en español (código HTTP 409) en lugar de un error de integridad. Tres casos concretos:

- No se puede eliminar un **proceso** con activos vivos asociados. Un activo ya eliminado lógicamente **no** bloquea.
- No se puede eliminar un **periodo** de riesgos que contenga riesgos vivos, ni el periodo activo.
- No se puede eliminar un **ítem de catálogo** que algún registro vivo esté usando. El camino correcto para retirar un valor en uso es **desactivarlo**, no borrarlo.

Es decir: en este sistema la regla de negocio la impone la aplicación, y `PROTECT` actúa como red de seguridad frente a un borrado físico (que, como se explicó en 3.0, desde SQL directo el motor rechaza igualmente).

## 3.2 CASCADE — inventario completo

| Hijo | Padre |
|---|---|
| `core_documentversion.document_id` | `core_document` |
| `core_matrixlevel.matrix_id` | `core_evaluationmatrix` |
| `core_riskrange.matrix_id` | `core_evaluationmatrix` |
| `core_riskevaluationasset.evaluation_id` | `core_riskevaluation` |
| `core_riskcontrol.line_id` | `core_riskevaluationasset` |
| `core_risktreatment.evaluation_id` | `core_riskevaluation` |
| `core_risktreatmentactivity.treatment_id` | `core_risktreatment` |
| `core_consenthistory.consent_id` | `core_consent` |
| `core_auditchecklistitem.audit_id` | `core_audit` |
| `core_auditfinding.audit_id` | `core_audit` |
| `core_auditactionplan.finding_id` | `core_auditfinding` |
| `core_incidentaction.incident_id` | `core_incident` |
| `core_incidentattachment.incident_id` | `core_incident` |
| `core_incidenthistory.incident_id` | `core_incident` |
| `core_arcoattachment.request_id` | `core_arcorequest` |
| `core_arcohistory.request_id` | `core_arcorequest` |
| `core_trainingquestion.training_id` | `core_training` |
| `core_trainingquestionoption.question_id` | `core_trainingquestion` |
| `core_trainingassignment.training_id` | `core_training` |
| `core_trainingassignment.user_id` | `auth_user` |

| Hijo | Padre |
|---|---|
| core_trainingexamattempt.assignment_id | core_trainingassignment |
| core_certificate.assignment_id | core_trainingassignment |
| core_systemcatalogitem.catalog_id | core_systemcatalog |
| core_userprofile.user_id | auth_user |
| core_roleprofile.group_id | auth_group |
| customer_domain.tenant_id | customer_client |
| customer_license.client_id | customer_client |

El criterio es uniforme: son **tablas hijas sin vida propia**. Una versión no existe sin su documento; una actividad no existe sin su control propuesto; un nivel de matriz no existe sin su matriz.

Las tres tablas puente de relaciones muchos a muchos (core_document_related_documents, core_document_gap_requirements, core_riskcontrol_annex_a_controls) también son CASCADE en ambos extremos, que es el comportamiento estándar para una tabla puente.

## 3.3 SET_NULL — inventario completo

| Columna | Apunta a | Por qué sobrevive el hijo |
|---|---|---|
| core_securitylog.user_id | auth_user | La bitácora no puede perder el asiento porque el usuario ya no exista. El nombre queda copiado en username. |
| core_consent.revoked_by_id | auth_user | La revocación es un hecho; quién la ejecutó es contexto. |
| core_consenthistory.changed_by_id | auth_user | Ídem, en el historial. |
| core_incidentattachment.uploaded_by_id | auth_user | El archivo sigue siendo evidencia. |
| core_incidenthistory.changed_by_id | auth_user | El evento ocurrió. |
| core_arcoattachment.uploaded_by_id | auth_user | Además puede nacer nulo: los adjuntos del formulario público no tienen usuario. |
| core_arcohistory.changed_by_id | auth_user | Ídem. |
| core_auditchecklistitem.gap_requirement_id | core_gaprequirement | Si se borra el requisito de origen, la auditoría conserva el ítem y su código copiado en source_requirement_code. |
| core_riskevaluation.carried_from_id | core_riskevaluation (autorreferencia) | Si se borra el riesgo del periodo anterior, la copia del periodo nuevo **no puede desaparecer con él**. |
| customer_platformlog.target_client_id | customer_client | La bitácora de plataforma sobrevive a la baja de una empresa. |
| core_personaldataoperation.created_by_id | auth_user | El hecho registrado sobrevive a la baja del usuario que lo capturó. |
| core_systemsettings.updated_by_id | auth_user | Los ajustes de la empresa siguen vigentes aunque se dé de baja a quien los cambió por última vez. *(Fila que faltaba en este inventario; la llave existe desde core/0023.)* |
| customer_platformaisettings.updated_by_id | auth_user | La configuración del asistente sigue vigente aunque se elimine al superadministrador que la guardó; el rastro de quién cambió qué queda además en customer_platformlog. |

**El patrón común:** cuando la referencia se pierde, el dato importante **se conserva copiado como texto** en la misma fila (`core_securitylog.username`, `core_auditchecklistitem.source_requirement_code`, `core_arcohistory.actor`). Es desnormalización deliberada, precisamente para que el borrado del padre no destruya la trazabilidad.

## 4. Restricciones de unicidad

### 4.1 Unicidad de una sola columna

| Tabla | Columna |
|---|---|
| `customer_client` | `schema_name` |
| `customer_domain` | `domain` |
| `core_process` | `code` |
| `core_document` | `code` |
| `core_asset` | `code` |
| `core_gaprequirement` | `requirement_code` |
| `core_annexacontrol` | `code` |
| `core_riskperiod` | `name` |
| `core_consent` | `consent_code` |
| `core_audit` | `audit_code` |
| `core_incident` | `incident_code` |
| `core_incidentrecipient` | `email` |
| `core_arcorequest` | `case_code` |
| `core_training` | `training_code` |
| `core_certificate` | `code` |
| `core_systemcatalog` | `code` |
| `core_userprofile` | `user_id` *(relación uno a uno)* |
| `core_roleprofile` | `group_id` *(relación uno a uno)* |
| `core_gapannexaassessment` | `annex_a_control_id` *(relación uno a uno)* |
| `core_riskcontrol` | `line_id` *(relación uno a uno)* |
| `core_certificate` | `assignment_id` *(relación uno a uno)* |
| `core_dashboardtarget` | `kpi_key` |
| `core_personaldataimpactassessment` | `asset_id` *(relación uno a uno)* |

### 4.2 Unicidad compuesta

| Nombre de la restricción | Tabla | Columnas | Qué garantiza |
|---|---|---|---|
| `unique_document_version` | `core_documentversion` | `document_id`, `version` | No puede haber dos versiones con el mismo número en un documento. |
| `unique_matrix_level` | `core_matrixlevel` | `matrix_id`, `axis`, `position` | Cada posición de cada eje de una matriz existe una sola vez. |
| `unique_risk_range` | `core_riskrange` | `matrix_id`, `level` | Cada matriz define una sola banda por nivel de severidad. |

| Nombre de la restricción | Tabla | Columnas | Qué garantiza |
|---|---|---|---|
| unique_risk_code_per_period | core_riskevaluation | period_id, risk_code | El código de riesgo es único **dentro de su periodo**, no globalmente. |
| unique_risk_evaluation_asset | core_riskevaluationasset | evaluation_id, asset_id | Un riesgo no puede vincularse dos veces al mismo activo. |
| unique_training_assignment | core_trainingassignment | training_id, user_id | Un participante se asigna una sola vez a una capacitación. |
| unique_systemcatalogitem_code_per_catalog | core_systemcatalogitem | catalog_id, code | El código de un ítem es único dentro de su catálogo; distintos catálogos pueden reutilizar códigos. |
| unique_dashboard_snapshot | core_dashboardsnapshot | month, kpi_key | No puede existir más de una foto del mismo indicador en el mismo mes; es lo que hace que la primera captura del mes «gane» sin necesidad de bloqueos. |

### 4.3 Restricciones únicas parciales — «solo uno activo»

Son dos, y merecen atención porque resuelven un problema recurrente **en la base de datos y no en el programa**: garantizar que exista como máximo un registro activo.

**unique_active_evaluation_matrix** sobre core_evaluationmatrix:

```
UNIQUE (status) WHERE (is_deleted = false AND status = 'A')
```

Solo una matriz de evaluación puede estar activa por empresa.

**unique_active_risk_period** sobre core_riskperiod:

```
UNIQUE (is_active) WHERE (is_active = true AND is_deleted = false)
```

Solo un periodo de evaluación de riesgos puede estar activo por empresa.

**Por qué en la base de datos y no en el programa.** Una comprobación en la aplicación («¿ya hay otra activa? entonces desactívala») es vulnerable a la concurrencia: dos peticiones simultáneas pueden leer «no hay ninguna activa» y activar las dos. La restricción parcial hace que la segunda transacción **falle**, siempre, con independencia del orden. La aplicación **además** desactiva la anterior al activar una nueva; la restricción es la garantía de último recurso.

Nótese que ambas condiciones incluyen is_deleted = false: una matriz o un periodo borrado lógicamente **no cuenta** como activo, aunque su fila siga en la tabla.

## 5. Índices

Además de los índices que PostgreSQL crea automáticamente para llaves primarias, restricciones de unicidad y columnas de llave foránea, el esquema declara **27 índices explícitos** sobre las columnas por las que se filtra y se ordena en las pantallas de listado:

| Tabla | Índice sobre |
|---|---|
| core_riskevaluation | controlled_severity; residual_severity; status |

| Tabla | Índice sobre |
|---|---|
| core_consent | status; granted_at; expires_at |
| core_audit | status; audit_type; start_date |
| core_auditfinding | classification |
| core_auditactionplan | due_date, status (compuesto) |
| core_incident | status; severity; incident_type; incident_date |
| core_arcorequest | status; right_type; priority; reception_datetime |
| core_training | status; start_date |
| core_trainingassignment | user_id, passed; training_id, passed (compuestos) |
| core_trainingexamattempt | assignment_id, taken_at (compuesto) |
| core_securitylog | created_at; user_id, created_at (compuesto) |
| customer_license | client_id, end_date (compuesto) |

Los compuestos responden a consultas concretas: `due_date, status` sirve al cálculo de acciones vencidas; `client_id, end_date` a la búsqueda de la licencia vigente de una empresa (la de mayor `end_date`); `user_id, created_at` a la consulta de la bitácora filtrada por usuario y ordenada por fecha.

## 6. Borrado lógico y su interacción con las restricciones UNIQUE

**Este apartado describe el comportamiento que más veces ha sorprendido en este sistema. Merece leerse completo.**

### 6.1 Cómo funciona el borrado lógico

De las 52 tablas, **19 incorporan un bloque común de siete columnas de auditoría**:

```
created_at    timestamptz NOT NULL,
created_by_id integer      NOT NULL REFERENCES auth_user(id),
updated_at    timestamptz NOT NULL,
updated_by_id integer      NULL     REFERENCES auth_user(id),
is_deleted    boolean     NOT NULL DEFAULT false,
deleted_at    timestamptz NULL,
deleted_by_id integer      NULL     REFERENCES auth_user(id)
```

Esas 17 tablas son: `core_gaprequirement`, `core_document`, `core_documentversion`, `core_process`, `core_asset`, `core_evaluationmatrix`, `core_appraisalrange`, `core_gapannexaassessment`, `core_riskperiod`, `core_riskevaluation`, `core_consent`, `core_audit`, `core_incident`, `core_arcorequest`, `core_training`, `core_systemcatalog` y `core_systemcatalogitem`. Son las **entidades cabecera** de cada módulo: aquellas que el usuario crea, edita y elimina directamente.

Cuando un usuario elimina un registro desde la aplicación, **no se ejecuta un `DELETE`**: se marca `is_deleted = true`, se anota la fecha y el usuario, y la fila permanece en la tabla. El sistema tiene dos vías de consulta:

- La vía **por defecto**, usada por toda la aplicación, añade automáticamente `WHERE is_deleted = false`. El registro desaparece de listados, búsquedas, informes y estadísticas.

- Una vía **explícita** que ve todas las filas, reservada para migraciones de datos, generación de códigos y tareas internas.

## 6.2 La consecuencia que hay que conocer

> **Las restricciones `UNIQUE` son de la base de datos y no saben nada del borrado lógico: también cuentan las filas marcadas como eliminadas.**

Es decir: si se elimina el activo con código `ACT-001`, ese código **sigue ocupado**. Un intento de crear otro activo con el mismo código chocará contra la restricción `UNIQUE`, aunque para el usuario el primero «ya no existe».

Esto es una consecuencia inevitable de combinar borrado lógico con unicidad física, no un defecto. Pero tiene efectos reales que el sistema aborda explícitamente en tres sitios:

1. **Mensaje de error comprensible.** Al intentar crear un ítem de catálogo cuyo código pertenece a una fila eliminada, la aplicación no devuelve un error técnico sino un texto en español que explica exactamente qué pasa: *«Ya existe un ítem con ese código en este catálogo (fue eliminado). Use otro código o restáurelo.»*
2. **Restauración en la carga masiva.** La importación de ítems de catálogo por archivo Excel, al encontrar un código ocupado por una fila eliminada, **la revive** en lugar de fallar, y lo reporta aparte como «restaurado».
3. **Generación de códigos correlativos.** Los generadores de códigos automáticos (`R-001`, `INC-001`, `PA-001`, `CERT-001`…) consultan **incluyendo las filas eliminadas**. Si contaran solo las visibles, reutilizarían un código ya reservado y provocarían un error de integridad.

Un cuarto caso se dio en el arrastre de riesgos entre periodos: si la copia de un riesgo en el periodo destino se eliminaba, su código quedaba reservado y el riesgo quedaba excluido para siempre del arrastre. La solución no fue relajar la comprobación —sin ella el error de integridad es inevitable— sino **revivir y reescribir** la fila eliminada cuando procede del mismo origen.

## 6.3 Tablas sin borrado lógico

Las 33 tablas restantes no llevan el bloque de auditoría, por tres motivos distintos:

- **Tablas hijas anidadas** (`core_matrixlevel`, `core_riskrange`, `core_riskcontrol`, `core_risktreatment`, `core_risktreatmentactivity`, `core_riskevaluationasset`, `core_auditchecklistitem`, `core_auditfinding`, `core_auditactionplan`, `core_incidentaction`, `core_trainingquestion`, `core_trainingquestionoption`…): se escriben y se reescriben junto con su padre, en bloque y dentro de una transacción. La auditoría la lleva el padre.
- **Tablas de historial inmutable** (`core_securitylog`, `core_consenthistory`, `core_incidenthistory`, `core_arcohistory`, `core_dashboardsnapshot`, `core_personaldataoperation`): no se borran nunca, ni lógica ni físicamente (apartado 7.2).
- **Tablas de configuración o control de plataforma** (`core_systemsettings`, `core_incidentrecipient`, `core_userprofile`, `core_roleprofile`, `core_annexacontrol`, y las cinco de `customer`).

# 7. Tablas con comportamiento especial

## 7.1 Tablas singleton — core_systemsettings y customer_platformaisettings

Guarda los interruptores que afectan a toda la empresa (habilitar el cálculo de Relevancia, qué activos entran al módulo de Riesgos, qué niveles de riesgo pasan a la bandeja de Tratamiento).

Su comportamiento se aparta de lo habitual en dos puntos, garantizados desde la capa de aplicación:

- Al grabar, **la llave primaria se fuerza siempre a 1**. Es imposible crear una segunda fila.
- **El borrado está bloqueado**: cualquier intento lanza un error. La fila no puede desaparecer, aunque su contenido sí se edite.

Se creó con valores por defecto en cada empresa mediante una migración de datos, y en las empresas nuevas nace con el esquema.

`customer_platformaisettings` **sigue exactamente el mismo patrón, pero en el esquema** `public` (creada por `customer/0004`): una sola fila **para toda la plataforma**, no una por empresa. Guarda la configuración del asistente de consultas teóricas —interruptor, proveedor, modelo y la clave de acceso al proveedor, cifrada—. La llave primaria se fuerza a 1, el borrado lanza un error, y la migración que crea la tabla siembra la fila con el asistente **apagado y sin clave**. El panel de administración no ofrece ni alta ni borrado: solo edición.

## 7.2 Tablas de historial inmutable

Seis tablas son **solo de adición**: admiten `INSERT`, pero tanto la modificación como el borrado están **bloqueados a nivel de modelo**, de modo que ni un administrador ni un error de programación pueden alterar el historial.

| Tabla | Qué registra |
|---|---|
| `core_securitylog` | Bitácora de seguridad de toda la empresa: altas, cambios, bajas, inicios de sesión, exportaciones. |
| `core_consenthistory` | Línea de tiempo de cada consentimiento (otorgado, revocado). |
| `core_incidenthistory` | Línea de tiempo de cada incidente. |
| `core_arcohistory` | Línea de tiempo de cada solicitud ARCO-P. |
| `core_dashboardsnapshot` | Foto mensual de 6 indicadores del Dashboard Ejecutivo. La primera captura del mes es definitiva. |
| `core_personaldataoperation` | Bitácora de hechos sobre datos personales de un activo (EIPD). Un error no se corrige editando: se corrige con una fila nueva que **anula** a la anterior (`annuls_id`). |

```
def save(self, *args, **kwargs):
    if self.pk is not None:                # ya existe → sería una modificación
        raise ValueError("SecurityLog es inmutable: no se permite modificar registros.")
    super().save(*args, **kwargs)

def delete(self, *args, **kwargs):
    raise ValueError("SecurityLog es inmutable: no se permite eliminar registros.")
```

> **Alcance real de esta garantía.** La protección vive en la capa de aplicación, **no en el motor**. Quien tenga acceso directo a PostgreSQL con privilegios de escritura puede modificar estas tablas. Si el nivel de aseguramiento exigido lo requiere, la

> medida complementaria es de permisos de base de datos y de acceso al servidor (apartado 4 del documento **05**), no de esquema.

### 7.3 Catálogos provisionados automáticamente

Tres conjuntos de datos **no los captura el usuario**: se crean al dar de alta la empresa y se rellenan en las existentes mediante migraciones de datos.

| Tabla | Contenido inicial | ¿Editable? |
|---|---|---|
| `core_annexacontrol` | Los **93 controles** del Anexo A de la norma ISO/IEC 27001:2022 (37 organizativos, 8 de personas, 14 físicos, 34 tecnológicos). | No. Catálogo de sistema. |
| `core_gapannexaassessment` | **93 filas**, una por control. | Sí, solo se edita; no se crea ni se elimina. |
| `core_evaluationmatrix` (+ niveles y bandas) | **3 matrices** predefinidas (3×3, 4×4, 5×5), con la 4×4 activa. | Sí, solo se edita y se activa. |
| `core_appraisalrange` | **4 franjas**: EXTREMO, ALTO, MEDIO, BAJO, con sus intervalos y colores. | Sí, se editan los valores; el nombre es fijo. |
| `core_systemcatalog` / `core_systemcatalogitem` | **7 catálogos** configurables. Los de amenazas (57 ítems) y vulnerabilidades (82) se entregan como **lista de arranque editable, no normativa**. | Los ítems sí; las cabeceras de catálogo solo por desarrollo. |
| `core_riskperiod` | Un **«Periodo inicial»** por empresa. | Sí. |
| `core_systemsettings` | Una fila con los valores por defecto. | Sí (solo edición). |

## 8. Tipos de datos empleados

| Tipo PostgreSQL | Uso en el sistema |
|---|---|
| `bigserial` | Todas las llaves primarias. |
| `bigint` | Las llaves foráneas hacia tablas de la aplicación. |
| `integer` (en llaves) | **Todas** las llaves foráneas hacia `auth_user` —la tabla nativa de usuarios usa una llave de 32 bits—: las de auditoría (`created_by_id`, `updated_by_id`, `deleted_by_id`) y el resto de referencias a un usuario (`user_id`, `approved_by_id`, `revoked_by_id`, `changed_by_id`, `uploaded_by_id`). Verificado en `pg_attribute` el 2026-10-15: 77 columnas, todas `integer`. |
| `varchar(n)` | Códigos, nombres, valores de lista. Las columnas de «lista» son `varchar(1)` o `varchar(2)` con códigos cortos (`'A'`, `'V'`, `'UA'`), traducidos a etiquetas en la interfaz. |
| `text` | Descripciones, justificaciones, evidencias y cualquier texto de longitud no acotada. |
| `integer` | Contadores, posiciones, niveles de madurez (0–5), escalas de probabilidad e impacto (1–N), porcentajes de avance. |
| `numeric(5,2)` | **Todo valor decimal de negocio**: valor de activo, puntajes de riesgo, cumplimiento, versiones de documento, puntajes de examen. Nunca coma flotante. |
| `boolean` | Indicadores. |
| `date` | Fechas sin hora (plazos, fechas compromiso). |
| `timestamptz` | Fechas con hora y zona. **Siempre se almacena en UTC.** |
| `time` | Hora del incidente y de su detección, separada de la fecha porque el formulario oficial las pide por separado. |
| `jsonb` | Selecciones múltiples y estructuras variables. |

| Tipo PostgreSQL | Uso en el sistema |
|---|---|
| `inet` | Direcciones IP (`core_securitylog.ip_address`, `core_consent.ip_address`). |
| `bytea` | Una sola columna: `customer_platformaisettings.api_key_encrypted`, que guarda **texto cifrado**. Es binario porque el contenido no es legible ni debe serlo. |

### 8.1 Dos criterios que conviene destacar

**Decimales exactos, nunca coma flotante.** Todos los valores calculados (`core_asset.asset_value`, `core_riskevaluation.controlled_score`, `core_gaprequirement.compliance`) usan `numeric(5,2)`. Un promedio de tres valores produce resultados como `4.67` o `4.33`, y la aritmética de coma flotante introduciría errores de redondeo en cifras que respaldan una evaluación de riesgo ante un auditor.

**`jsonb` solo donde la alternativa sería una tabla vacía de contenido.** Se usa en dos situaciones concretas:

- **Selecciones múltiples contra catálogos fijos**: `core_incident.affected_data_types`, `core_incident.security_measures_adopted`, `core_incident.affected_cid`, `core_asset.data_categories`, `core_systemsettings.risk_treatment_levels`. Crear una tabla puente para una lista de códigos que solo se lee completa, junto con su fila padre, no aportaría nada. La validación de que cada código pertenece a su catálogo se hace en la capa de aplicación.
- **Estructuras de contexto variable**: `core_securitylog.detail`, `customer_platformlog.detail`, `core_riskperiod.snapshot_risk_ranges`, `core_trainingexamattempt.answers`.

## 9. Tablas nativas de Django

Las siguientes tablas las gestiona el marco de trabajo y **conservan su estructura estándar**, sin modificaciones:

| Tabla | Contenido | Dónde vive |
|---|---|---|
| `auth_user` | Usuarios: nombre de usuario, contraseña (siempre en forma de resumen criptográfico, nunca en claro), nombre, correo, indicadores `is_active` / `is_staff` / `is_superuser`. | **En `public`** (superadministradores de Gestitel) **y en cada esquema de empresa** (usuarios de esa empresa). Son tablas distintas. |
| `auth_group` | Los **roles**. | Ídem. |
| `auth_permission` | Los **permisos** disponibles, uno por acción y modelo (`view_asset`, `add_document`, `change_incident`, `delete_process`…). | Ídem. |
| `auth_user_groups` | Qué rol tiene cada usuario. | Ídem. |
| `auth_group_permissions` | Qué permisos otorga cada rol. | Ídem. |
| `auth_user_user_permissions` | Permisos asignados directamente a un usuario. | Ídem. |
| `django_content_type` | Registro de los modelos, requerido por el sistema de permisos. | Ídem. |
| `django_migrations` | Historial de migraciones aplicadas. Es lo que permite saber en qué versión de esquema está cada empresa. | En cada esquema. |
| `django_session` | Sesiones del panel de administración. | En `public`. |
| `django_admin_log` | Acciones realizadas desde el panel de administración de Django. | En `public`. |

Dos extensiones propias se apoyan en estas tablas sin reemplazarlas: `core_userprofile` (relación uno a uno con `auth_user`) y `core_roleprofile` (relación uno a uno con `auth_group`).

## 10. Evolución del esquema

El esquema **no se modifica manualmente**. Todo cambio se expresa como una **migración**: un archivo versionado en el repositorio de código, numerado y con dependencias explícitas.

| Aplicación | Migraciones | Ámbito |
|---|---|---|
| `customer` | `0001` – `0003` | Esquema `public` |
| `core` | `0001` – `0043` | Todos los esquemas de empresa |

Las migraciones son de dos clases:

- **De esquema** — crean o alteran tablas, columnas, restricciones e índices.
- **De datos** (`RunPython`) — transforman o siembran contenido. De las 43 migraciones de `core`, **29 contienen al menos un paso de datos**.

Se aplican con:

```
python manage.py migrate_schemas --shared   # solo el esquema public
python manage.py migrate_schemas            # public + todos los esquemas de empresa
```

El detalle de qué hace cada migración, cuáles son destructivas y cuáles reversibles está en el documento **04 — Procedimientos SQL y scripts**, apartado 5.

### Documentos relacionados

| Documento | Contenido |
|---|---|
| **01 — Diagramas entidad-relación** | Mapa visual por dominio funcional. |
| **02 — Diccionario de datos** | Las 52 tablas columna por columna. |
| **04 — Procedimientos SQL y scripts** | Dónde vive la lógica de negocio; scripts y migraciones. |
| **05 — Políticas de acceso y seguridad** | Quién puede ver, modificar o borrar cada dato. |